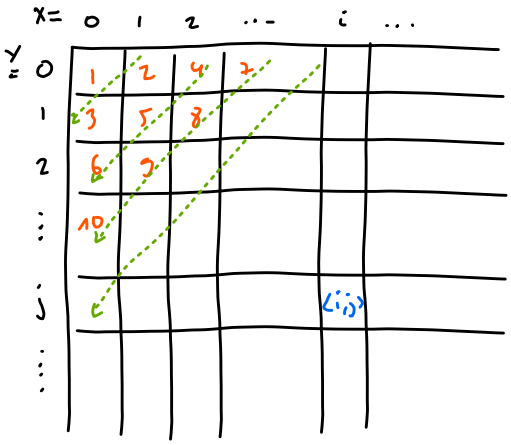
TheoInf1
Woche 10

Paare in Loop

Speichern von Paaren ⟨x, y⟩ von zwei Zahlen x, y ∈ $\mathbb{N}_0$ in einer einzelnen Zahl p ∈ $\mathbb{N}$.

Forderung: Unterschiedliche Paare werden als unterschiedliche Zahlen kodiert!

▶ **1. Möglichkeit:** "Diagonales Aufzählen"



IN DER ROTEN REIHENFOLGE WERDEN DIE PAARE AUFGEZÄHLT

JEDES FELD IST EIN PAAR

die roten Zahlen kodieren dann das Paar, z.B. ⟨1, 2⟩ = 9
Nachteil: evtl. recht schwer auszurechnen

▶ **2. Möglichkeit:** Als Exponenten von Primfaktoren
$p = 2^x \cdot 3^y$ ist der Code für das Paar ⟨x, y⟩

Beispiele:
$\langle 1, 2\rangle = 2^1 \cdot 3^2 = 18$
$\langle 0, 3\rangle = 2^0 \cdot 3^3 = 27$

Die erste Zahl (also x) soll man mit first(p) bekommen, die zweite Zahl (also y) mit second(p):

x ← first(p)

```
x ← 0
if p > 0
   h ← p
   loop p
      if h % 2 = 0
         ++x
         h ← h / 2
      end
   end
end
```

y ← second(p)

```
y ← 0
if p > 0
   h ← p
   loop p
      if h % 3 = 0
         ++y
         h ← h / 3
      end
   end
end
```

Anmerkung:
Der Wert 0 entspricht keinem Paar, aber wir programmieren es so, dass first(0) = second(0) = 0

Man kann natürlich auch die Werte im Paar getrennt verändern:

first(p) ← x

```
y ← second(p)
p ← (2 ^ x) · (3 ^ y)
```

second(p) ← y

```
x ← first(p)
p ← (2 ^ x) · (3 ^ y)
```

Notationszucker:

p ← ⟨x, y⟩

```
p ← (2 ^ x) · (3 ^ y)
```

⟨x, y⟩ ← p

```
x ← first(p)
y ← second(p)
```

Listen in Loop

Sei $n \in \mathbb{N}$ und $a_0, a_1, \ldots, a_{n-1} \in \mathbb{N}_0$ eine Reihe von n Zahlen.
Wir speichern die Liste $[a_0, a_1, \ldots, a_{n-1}]$ in einer einzelnen Zahl $a \in \mathbb{N}_0$:

Der Witz: wie lang ist die Liste ist, kann während des Loop-Programms geändert werden.

z.B.
n = 0: [ ] ⇒ a = 0
n = 1: $[a_0] \Rightarrow a = \langle a_0, 0\rangle$
n = 2: $[a_0, a_1] \Rightarrow a = \langle a_0, [a_1]\rangle = \langle a_0, \langle a_1, 0\rangle\rangle$
n = 3: $[a_0, a_1, a_2] \Rightarrow a = \langle a_0, [a_1, a_2]\rangle = \left\langle a_0, \langle a_1, \langle a_2, 0\rangle\rangle\right\rangle$

beliebiges n: $[a_0, a_1, \ldots., a_{n-1}] = \langle a_0, [a_1, \ldots, a_{n-1}]\rangle$

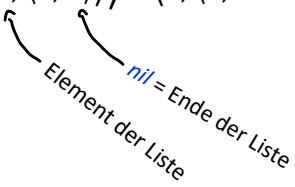
$= \left\langle a_0, \left\langle a_1, \left\langle a_2, \ldots\langle a_{n-2}, \langle a_{n-1}, 0\rangle\rangle\ldots\right\rangle\right\rangle\right\rangle$

0 ist das Zeichen für das Ende der Liste: 0 selbst kann ja kein Paar sein.
(Das Zeichen für das Ende der Liste nennt man auch *nil*)

0 ist auch der Code für die leere Liste [ ]

▸ Beispiel:

$[3, 0, 4] = \langle 3, \langle 0, \langle 4, 0\rangle\rangle\rangle = \langle 3, \langle 0, 2^4 \cdot 3^0\rangle\rangle = \langle 3, 2^0 \cdot 3^{16}\rangle = 2^3 \cdot 3^{3^{16}}$



▸ Länge der Liste

c ← count(a)

```
c ← 0
h ← a
loop a
   if h > 0
      ++c
      h ← second(h)
   end
end
```

▸ Zugriff auf das i-te Element ("random access")

für $i \in \mathbb{N}_0$ kann man das i-te Element aus der Liste a extrahieren. Dazu definieren wir erst einmal die Hilfsfunktion "rest": $rest(a, i) = [a_i, a_{i+1}, \ldots, a_{n-1}]$

$x \leftarrow rest(a, i)$

```
x ← a
loop i
   x ← second(x)
end
```

$x \leftarrow a_i$

```
x ← rest(a, i)
x ← first(x)
```

$a_i \leftarrow x$

```
h ← rest(a, i+1)
h ← ⟨x, h⟩
if i > 1
   for j ← 1, ..., i
      h ← ⟨a_{i-j}, h⟩
   end
end
a ← h
```

$h \leftarrow [a_{i+1}, \ldots, a_{n-1}]$

$h \leftarrow [x, a_{i+1}, \ldots, a_{n-1}]$

Goto-Sprache

Eine neue Variante von Loop und While, deren Funktionsweise Ähnlich zu der von CPUs ist.

▸ Syntax

⟨Programm⟩ ::= ε | ⟨Befehl⟩ "↲" ⟨Programm⟩ | ⟨Label⟩ ":" ⟨Befehl⟩ "↲" ⟨Programm⟩
⟨Befehl⟩ ::= "++" ⟨Variable⟩
| "−−" ⟨Variable⟩
| "*if*" ⟨Variable⟩ "↲" ⟨Programm⟩ "*end*"
| "*goto*" ⟨Label⟩
| "*stop*"

⟨Variable⟩ und ⟨Label⟩: Strings aus Zahlen und Buchstaben, aber nicht z.B. "loop", "end", "stop", "goto", "if", …

if x ⟨P⟩ *end*	Führt das Programm ⟨P⟩ aus, wenn x > 0 ist.
goto ⟨Label⟩	Breche hier den Programmablauf ab und mache mit der Befehlszeile weiter, die mit ⟨Label⟩ markiert ist.

▸ Regeln für Label:

- Für jedes Label darf es im Programm nur eine Programmzeile geben, die mit diesem Label markiert ist ("markiert" heißt, das Label steht vor dem Befehl in dieser Zeile: ⟨Label⟩ ":" ⟨Befehl⟩)
- Jedes Label, das von einem *goto* verwendet wird, muss auch einmal im Programm vor einem Befehl vorkommen.

▸ Goto kann *loop*

loop x
⟨P⟩
end

```
    h ← x
L:  if h > 0
      ⟨P⟩
      −−h
      goto L
    end
```

h darf in ⟨P⟩ nicht vorkommen

▸ Goto kann *while*

while x
⟨P⟩
end

```
L:  if x > 0
      ⟨P⟩
      goto L
    end
```

▸ Extended Goto

Bis auf *if* und *if-else* kann alles aus extended Loop (und aus extended While) für extended Goto übernommen werden. 🙄

▸ *else* für Goto

```
if x
    ⟨P1⟩
else
    ⟨P2⟩
end
```

```
h ← x
if h
    ⟨P1⟩
end
if not h
    ⟨P2⟩
end
```

h darf nicht in ⟨P1⟩ oder ⟨P2⟩ vorkommen